STUDY NOTES · BEGINNER LEVEL

How Neural Networks Actually Work

A plain-English walkthrough of 3Blue1Brown's "Neural Networks" series — built for intuition first, math second.

> Based on the 4-part YouTube series by 3Blue1Brown (Grant Sanderson): *"But what is a neural network?"*, *"Gradient descent, how neural networks learn"*, *"What is backpropagation really doing?"*, and *"Backpropagation calculus"*. The running example throughout is a network that reads a handwritten digit (0–9) from an image.

01 The Structure — Neurons, Layers, Weights

02 Gradient Descent — How Learning Happens

03 Backpropagation — The Intuition

04 Backpropagation — The Actual Math

— Glossary of Terms

CHAPTER 01

The Structure — Neurons, Layers, Weights

The example used throughout: a network that looks at a 28×28 pixel image of a handwritten digit and guesses which number (0–9) it is.

What is a "neuron"?

- A neuron is **not** a brain cell here. It is just a box that holds one number.
- That number is called its **activation**, and it usually sits between 0 and 1.
- Think of activation as "how lit up" the neuron is — 0 means off, 1 means fully on.

> *Intuition: a neuron is like a dimmer switch, not an on/off switch. It reports a brightness level, not just yes/no.*

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

What is a "layer"?

- A layer is just a column of neurons, stacked side by side.
- **Input layer** — one neuron per pixel of the image (784 neurons for a 28×28 image). Each holds the pixel's brightness.
- **Hidden layers** — the middle layers, where the actual "thinking" happens. Not directly seen by the input or the final answer.
- **Output layer** — one neuron per possible answer (10 neurons for digits 0–9). The brightest one is the network's guess.

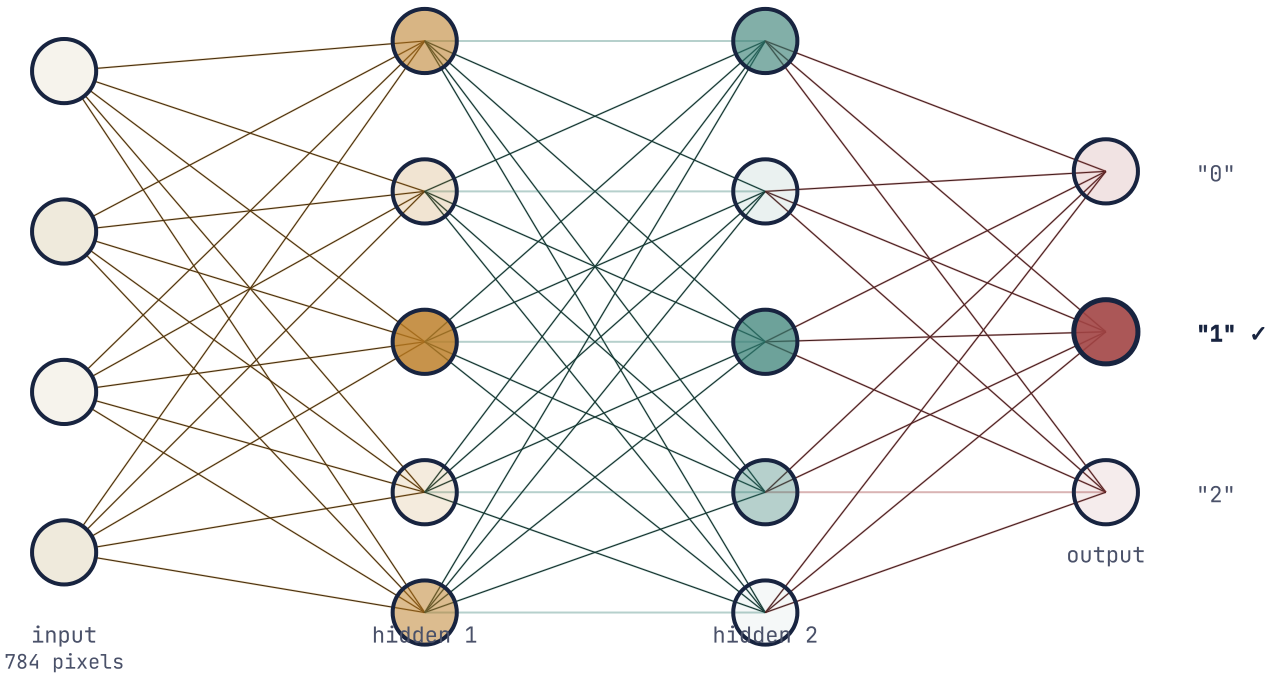


Fig 1 — A simplified network. Real digit-recognizer has 784 input neurons and 10 output neurons; only a few are drawn. Brighter fill = higher activation. The brightest output neuron is the guess.

How does one neuron get its brightness (activation)?

- Every neuron in a layer is connected to every neuron in the layer before it.
- Each connection has a number attached called a **weight** — it says how much that connection matters, and can be positive (excites) or negative (suppresses).
- A neuron's activation is a **weighted sum** of all the previous layer's activations, plus one extra number called the **bias**.
- The bias is a threshold — it makes a neuron harder or easier to activate, regardless of the input.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- That raw weighted sum can be any number (very negative to very positive), so it gets squashed into a 0–1 range using an **activation function**.

```
z = (w₁·a₁ + w₂·a₂ + ... + wₙ·aₙ) + b new activation = σ(z)
```

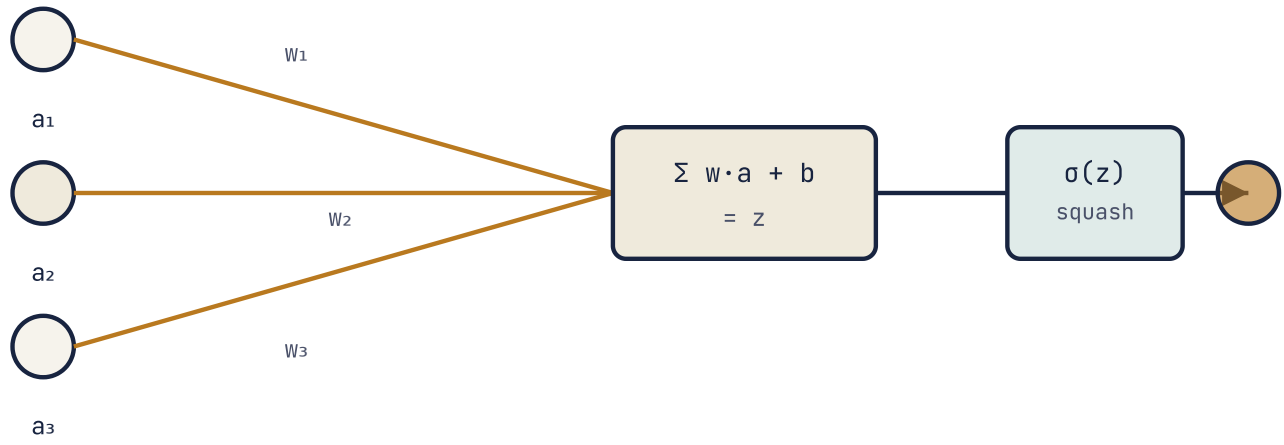


Fig 2 — Zoomed into one neuron: multiply each incoming activation by its weight, add them up, add the bias, then squash the result through an activation function.

The activation function — why we "squash" the number

- **Sigmoid** — the classic choice. Smoothly squishes any number into the range (0, 1). Very negative → near 0. Very positive → near 1.
- **ReLU** (Rectified Linear Unit) — modern default. Simpler rule: if the number is negative, output 0. Otherwise, pass it through unchanged.
- ReLU is preferred in practice mainly because it is easier and faster to train — 3Blue1Brown notes early networks used sigmoid mostly for historical, biology-inspired reasons.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

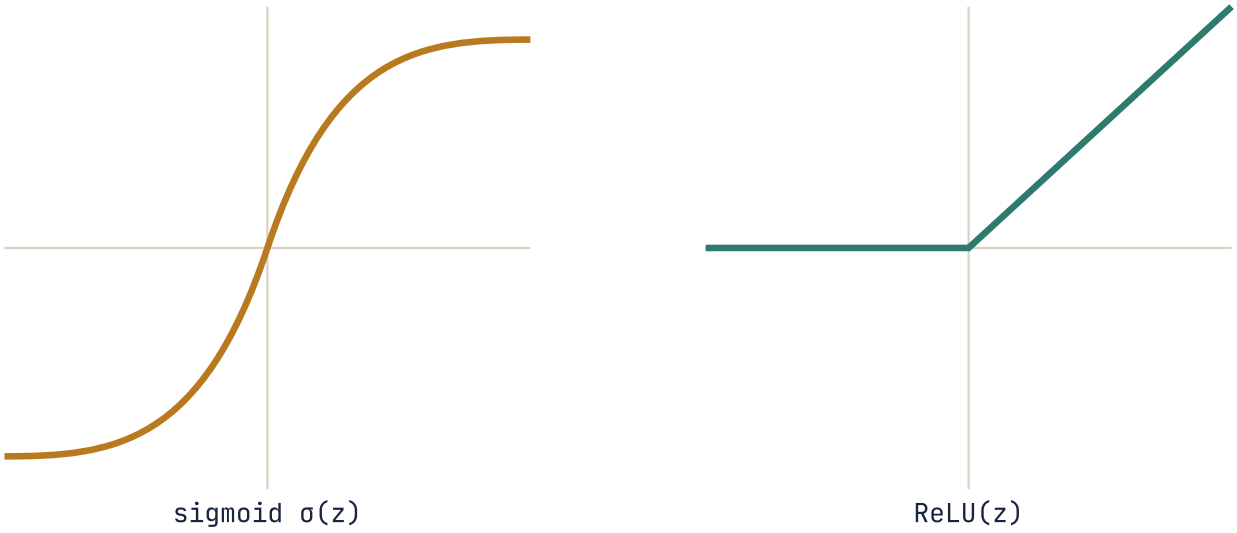


Fig 3 — Sigmoid smoothly bends into a ceiling and floor. ReLU is a flat zero, then a straight line up. Both stop "very negative" numbers from doing damage.

Putting it together

- The whole network is just: **a pile of weights and biases** connecting layer to layer.
- For the digit example, that's roughly **13,000 tunable numbers** (weights + biases) across all connections.
- "Learning" a neural network means finding good values for all of these numbers — nothing more mystical than that.
- Before training, weights and biases are random, so the network's guesses are random too.

CAUTION — WHAT THE NETWORK IS NOT DOING

3Blue1Brown is careful to point out: we would like to believe the hidden layers learn to detect edges, then loops, then shapes — the way a human might reason. But when you actually inspect a trained network's weights, they often look like noisy, non-interpretable patterns. The network finds whatever numbers reduce error — not necessarily concepts a human would recognize.

CHAPTER 02

Gradient Descent — How Learning Happens

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 1: Measure how wrong the network is

- Feed the network a training image where you already know the correct answer.
- Compare the network's 10 output activations to what they *should* be (1 for the correct digit, 0 for all others).
- Add up the squared differences — this single number is the **cost** (also called loss) for that one example.
- Low cost = good guess. High cost = bad guess.

```
cost = (output₀ - target₀)² + (output₁ - target₁)² + ... + (output₉ - target₉)²
```

- Average this cost over thousands of training examples → one overall number describing "how bad is this network right now."

Step 2: Think of the cost as a landscape

- Imagine every one of the ~13,000 weights and biases as a knob you can turn.
- Turning the knobs changes the cost. So the cost is like a landscape with hills and valleys, with one "input" per knob.
- Impossible to picture in 13,000 dimensions — so 3Blue1Brown simplifies it to a landscape with just **one** input, shaped like a curve, to build intuition.
- **Goal:** find the knob settings that land in the lowest valley — the settings that make the cost as small as possible.

> *Intuition: imagine you're standing on a hilly landscape in thick fog. You can't see the whole map — you can only feel the slope right under your feet. The sensible strategy: take a small step in whichever direction goes downhill, then repeat.*

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

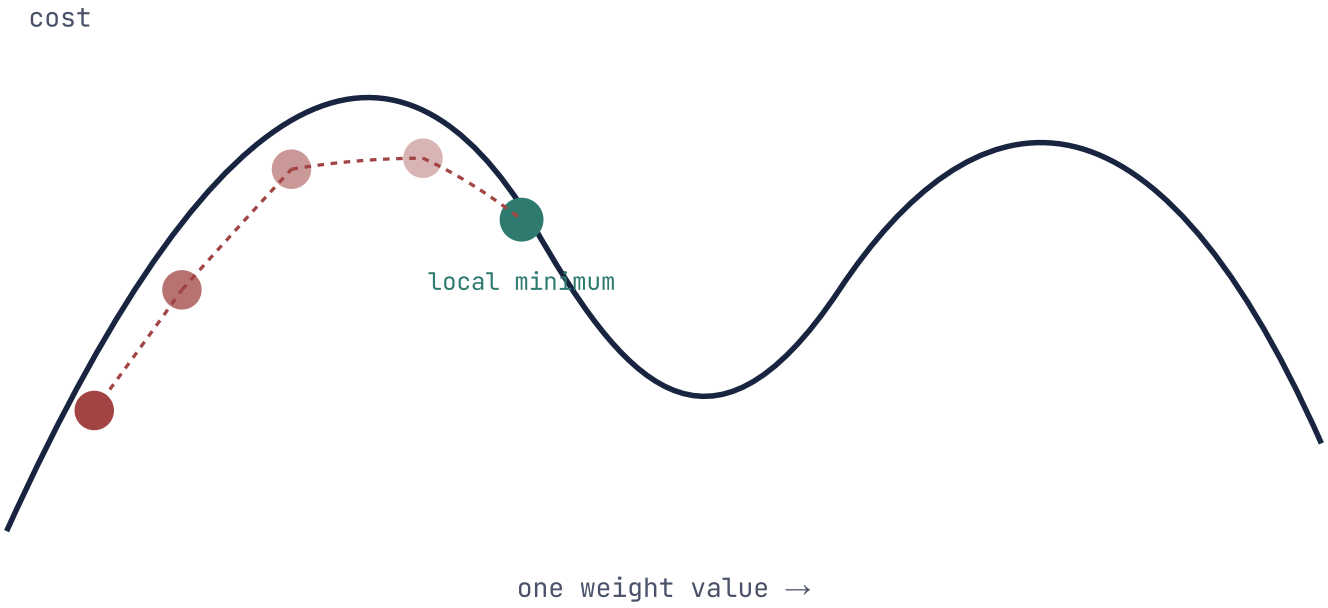


Fig 4 — Gradient descent: each red dot is one step. Step size shrinks automatically as the slope flattens near the bottom. Note it settled into *a* valley — not necessarily the lowest possible valley.

Step 3: The "gradient" — which way is downhill?

- In calculus, the **gradient** of a function points in the direction of *steepest increase*.
- So to go downhill fastest, move in the **opposite** direction of the gradient.
- The gradient isn't just a direction — its size also tells you: which knobs matter most right now, and by how much each one should shift.

```
new_weights = old_weights - (learning_rate × gradient)
```

- **Learning rate** — controls step size. Too big → overshoot and bounce around. Too small → learning crawls painfully slowly.
- This whole repeated nudging process is called **gradient descent**: the core algorithm behind almost all neural network training.

A shortcut: Stochastic Gradient Descent

- Computing the true gradient needs to check *every* training example — extremely slow with large datasets.
- Shortcut: split training data into small random batches ("mini-batches"), compute an approximate gradient from just one batch, and step using that.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- It's less precise per step, but far faster overall — like a drunk person stumbling downhill quickly rather than a careful surveyor calculating the exact slope every time.

KEY TAKEAWAY

"Learning" = minimizing a cost function. Nothing about it guarantees the network truly "understands" digits — it's just an optimization process finding numbers that reduce prediction error on the training data.

CHAPTER 03

Backpropagation — The Intuition

Gradient descent tells us *what to do* (go downhill). Backpropagation is the algorithm that actually *computes* the gradient — i.e., figures out how much to nudge every single weight and bias.

Starting from one training example

- Say you show the network an image of a handwritten "2", but it currently guesses "8" is more likely.
- Look at the output layer: the "2" neuron should get brighter, and all other neurons (including "8") should get dimmer.
- The size of the desired nudge matters too: a neuron that's very wrong needs a bigger push than one that's only slightly wrong.

Three ways to change a neuron's activation

To make the "2" output neuron brighter, three levers are available (recall: activation = σ(weighted sum + bias)):

1. **Increase the bias** — directly makes the neuron easier to activate.
2. **Increase the weights** — especially the weights connected to previous-layer neurons that are already bright (since brighter neurons have more influence).
3. **Change the previous layer's activations** — make the neurons that positively influence this output brighter, and the ones that negatively influence it dimmer.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

> *Intuition: think of it as neurons "voting." If neuron A has a strong positive weight into the "2" neuron, then the "2" neuron effectively says: "I wish neuron A were brighter" — like sending a request backward.*

Propagating the "wish list" backward

- The output layer doesn't just want changes in its own weights and biases — it also has a **wish list** for the previous layer's activations.
- But an activation can't be edited directly — it's only decided by ITS OWN weights, biases, and the layer before it.
- So the "wish" for that neuron gets translated into wishes for *its* weights, bias, and the layer before *that*.
- This is why it's called **backpropagation**: desired changes are passed backward, layer by layer, all the way to the input.

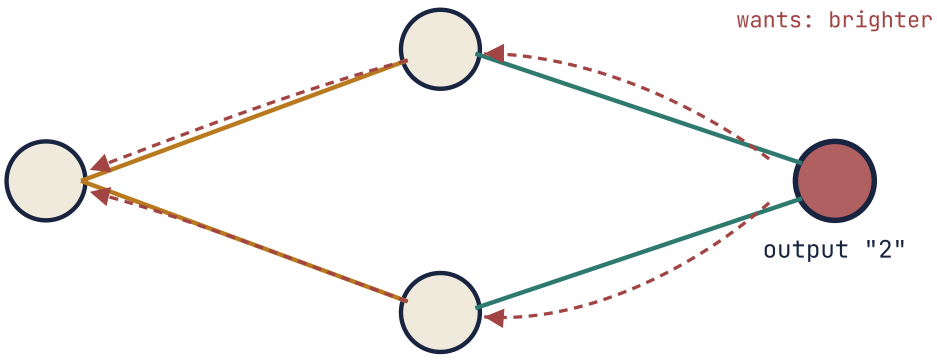


Fig 5 — The output neuron's desire to change ripples backward: first to the weights/bias feeding it, then to what the previous layer "should" look like, and so on toward the input.

Averaging across all examples

- Every training example generates its own wish list for how weights and biases should change.
- Average the wish lists across a batch of examples — this average *is* (an approximation of) the negative gradient used in gradient descent.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Nudge every weight and bias a small amount in that averaged direction, then repeat with a new batch.

CHAPTER 04

Backpropagation — The Actual Math

Chapter 3 gave the intuition in words. This chapter turns "wishes" into precise numbers using the **chain rule** from calculus.

Zoom into the simplest possible case

- Imagine a tiny network: one neuron per layer, connected in a straight line.
- Let `L` be the last layer. The neuron in layer L has activation `aᴸ`, computed from the previous layer's activation `aᴸ⁻¹`, a weight `w`, and a bias `b`.

```
zᴸ = w·aᴸ⁻¹ + b aᴸ = σ(zᴸ) Cost C₀ = (aᴸ - target)²
```

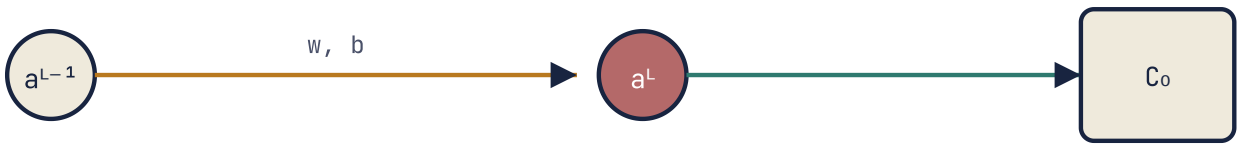


Fig 6 — A one-neuron-per-layer chain. Each arrow is a small function; the chain rule lets us ask "how much does a tiny nudge to w ripple all the way to the cost C₀?"

The chain rule, in plain words

- Question: "If I nudge the weight `w` a tiny bit, how much does the final cost `C₀` change?"
- Answer using the chain rule: multiply together the sensitivity of each step in the chain.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
∂C₀/∂w = ∂zᴸ/∂w × ∂aᴸ/∂zᴸ × ∂C₀/∂aᴸ
```

- `∂zᴸ/∂w` — how much the weighted sum changes per unit change in w. Turns out to equal `aᴸ⁻¹` (a bigger incoming activation means the weight matters more).
- `∂aᴸ/∂zᴸ` — how much the activation function's output changes as its input changes (the slope of sigmoid or ReLU at that point).
- `∂C₀/∂aᴸ` — how much the cost changes as the final activation changes. Turns out to equal `2(aᴸ - target)`.

> *Intuition: it's a relay race of sensitivity. "How much does w affect z?" times "how much does z affect a?" times "how much does a affect the cost?" — multiply the three effects together to get the total effect of w on the cost.*

Why this matters for real (multi-neuron) networks

- Real layers have many neurons, so the same idea applies but every activation depends on *all* neurons in the previous layer, not just one.
- The derivative of cost with respect to an earlier activation becomes a **sum** over every path that activation influences downstream.
- This is exactly the backward "wish list" from Chapter 3 — now expressed as precise partial derivatives instead of words.
- Stack these partial derivatives for every weight and bias in the network → that full collection *is* the gradient used in gradient descent.

SYMBOL	PLAIN MEANING
∂C/∂w	How much a tiny change in this weight moves the total cost
∂C/∂b	How much a tiny change in this bias moves the total cost
∂C/∂a (previous layer)	How much this network "wishes" the earlier neuron were different

KEY TAKEAWAY

Backpropagation is just the chain rule applied efficiently, computed backward from the output toward the input, so each layer's derivatives can reuse work already done for the layer after it. That efficiency is exactly what makes training large networks computationally feasible.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

REFERENCE

Glossary of Terms

Neuron
A unit that holds a single number (its activation), computed from the previous layer.

Activation
The number stored in a neuron, usually between 0 and 1.

Weight
A number attached to a connection between two neurons; controls how strongly one influences the other.

Bias
An extra number added to a neuron's weighted sum; shifts how easily that neuron activates.

Activation function
A function (like sigmoid or ReLU) that squashes the weighted sum into a usable range.

Cost function (loss)
A number measuring how wrong the network's output is compared to the correct answer.

Gradient descent
An algorithm that repeatedly nudges all weights and biases slightly downhill on the cost landscape.

Gradient
The direction (and size) of steepest increase in the cost; we step in the opposite direction.

Learning rate
How big a step gradient descent takes on each update.

Backpropagation
The algorithm that computes the gradient by passing "desired changes" backward through the network using the chain rule.

Chain rule
A calculus rule for finding how a change at one point in a chain of functions ripples through to the final output.

Mini-batch / Stochastic Gradient Descent

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Estimating the gradient using a small random sample of training data instead of the whole dataset, for speed.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF